

從原型設計到正式環境：超參數調整

來源：<https://codelabs.developers.google.com/vertex-p2p-hyperparameter?hl=zh-tw>

步驟數：6

目錄

1. [總覽](#)
2. [Vertex AI 簡介](#)
3. [設定環境](#)
4. [將訓練應用程式程式碼容器化](#)
5. [使用 SDK 執行超參數調整工作](#)
6. [清除所用資源](#)

步驟 1 · 約 1 分鐘

1. 總覽

在本實驗室中，您將使用 [Vertex AI](#) 在 Vertex AI 訓練上執行超參數調整工作。

本實驗室是「從原型設計到投入正式環境」系列影片的一部分。請務必先完成[上一個實驗室](#)，再進行這個實驗室。如要瞭解詳情，請觀看隨附的影片系列：



。

課程內容

內容如下：

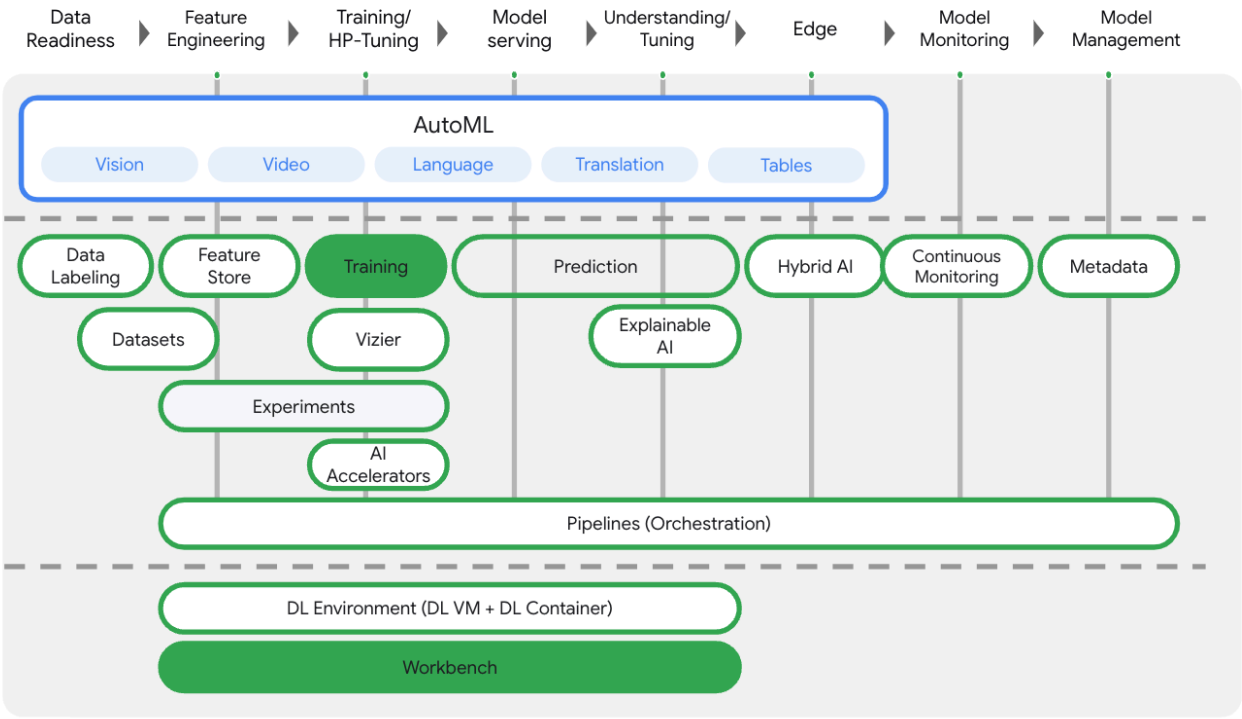
- 修改訓練應用程式程式碼，進行自動超參數調整
- 使用 [Vertex AI Python SDK](#) 設定及啟動超參數調整工作

在 Google Cloud 上執行這個實驗室的總費用約為 **\$1 美元**。

2. Vertex AI 簡介

本實驗室使用 Google Cloud 最新推出的 AI 產品服務。Vertex AI 整合了 Google Cloud 機器學習服務，提供流暢的開發體驗。以 AutoML 訓練的模型和自訂模型，先前需透過不同的服務存取。這項新服務將兩者併至單一 API，並加入其他新產品。您也可以將現有專案遷移至 Vertex AI。

Vertex AI 包含許多不同的產品，可支援端對端機器學習工作流程。本實驗室將著重於下列產品：訓練和 Workbench



步驟 3 · 約 10 分鐘

3. 設定環境

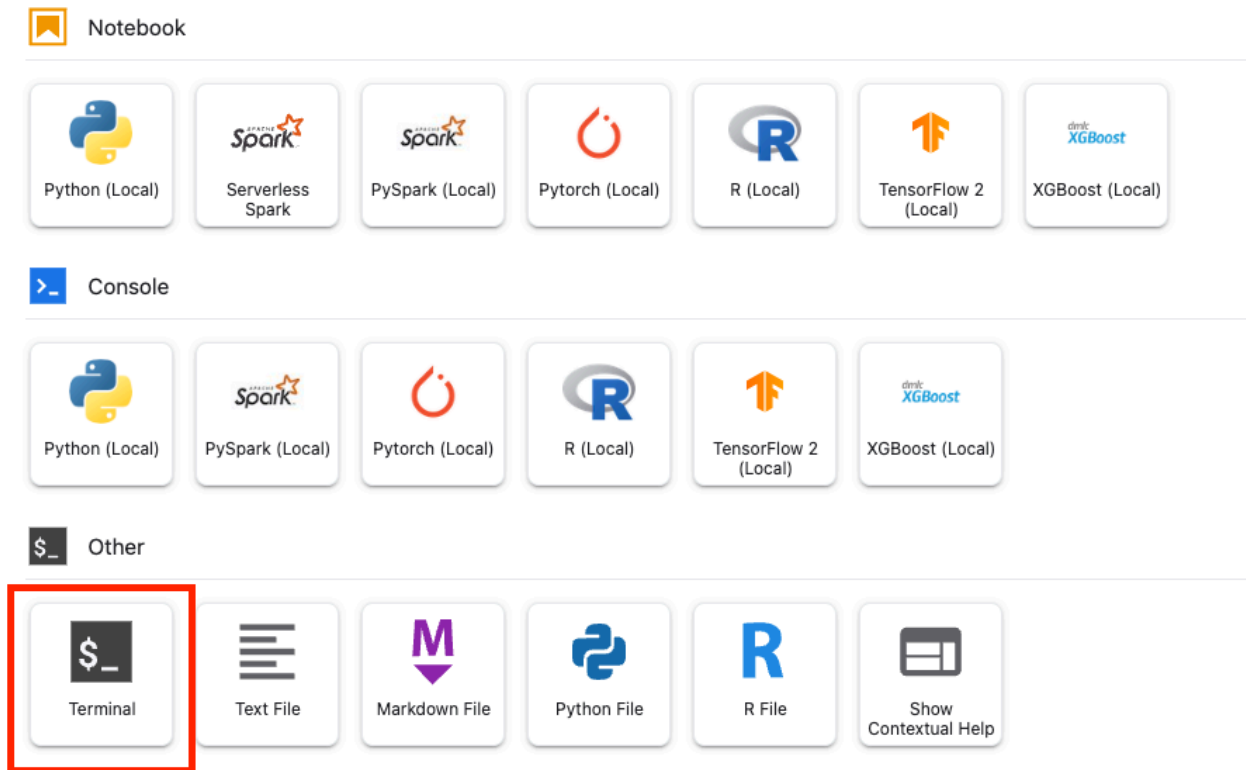
完成「[使用 Vertex AI 訓練自訂模型](#)」實驗室中的步驟，設定環境。

步驟 4 · 約 10 分鐘

4. 將訓練應用程式程式碼容器化

您要將訓練應用程式程式碼放入 [Docker 容器](#)，並將這個容器推送至 [Google Artifact Registry](#)，藉此將訓練工作提交至 Vertex AI。使用這種方法，您可以訓練及調整以任何架構建構的模型。

首先，請從先前實驗室建立的 Workbench 筆記本的啟動器選單中，開啟終端機視窗。



步驟 1：編寫訓練程式碼

建立名為 `flowers-hptune` 的新目錄，然後切換至該目錄：

```
mkdir flowers-hptune
cd flowers-hptune
```

執行下列指令，為訓練程式碼建立目錄，以及您將在其中新增下列程式碼的 Python 檔案。

```
mkdir trainer
touch trainer/task.py
```

`flowers-hptune/` 目錄現在應該包含下列項目：

```
+ trainer/
  + task.py
```

接著，開啟剛才建立的 `task.py` 檔案，然後複製下列程式碼。

請將 `BUCKET_ROOT` 中的 `{your-gcs-bucket}` 改成您在 [實驗室 1](#) 中儲存花卉資料集的 Cloud Storage bucket。

```

import tensorflow as tf
import numpy as np
import os
import hypertune
import argparse

## Replace {your-gcs-bucket} !!
BUCKET_ROOT='/gcs/{your-gcs-bucket}'

# Define variables
NUM_CLASSES = 5
EPOCHS=10
BATCH_SIZE = 32

IMG_HEIGHT = 180
IMG_WIDTH = 180

DATA_DIR = f'{BUCKET_ROOT}/flower_photos'

def get_args():
    '''Parses args. Must include all hyperparameters you want to tune.'''

    parser = argparse.ArgumentParser()
    parser.add_argument(
        '--learning_rate',
        required=True,
        type=float,
        help='learning rate')
    parser.add_argument(
        '--momentum',
        required=True,
        type=float,
        help='SGD momentum value')
    parser.add_argument(
        '--num_units',
        required=True,
        type=int,
        help='number of units in last hidden layer')
    args = parser.parse_args()
    return args

def create_datasets(data_dir, batch_size):
    '''Creates train and validation datasets.'''

    train_dataset = tf.keras.utils.image_dataset_from_directory(
        data_dir,
        validation_split=0.2,
        subset="training",
        seed=123,
        image_size=(IMG_HEIGHT, IMG_WIDTH),
        batch_size=batch_size)

```

```

validation_dataset = tf.keras.utils.image_dataset_from_directory(
    data_dir,
    validation_split=0.2,
    subset="validation",
    seed=123,
    image_size=(IMG_HEIGHT, IMG_WIDTH),
    batch_size=batch_size)

train_dataset = train_dataset.cache().shuffle(1000).prefetch(buffer_size=tf.data.AUTOTUNE)
validation_dataset = validation_dataset.cache().prefetch(buffer_size=tf.data.AUTOTUNE)

return train_dataset, validation_dataset

def create_model(num_units, learning_rate, momentum):
    '''Creates model.'''

    model = tf.keras.Sequential([
        tf.keras.layers.Resizing(IMG_HEIGHT, IMG_WIDTH),
        tf.keras.layers.Rescaling(1./255, input_shape=(IMG_HEIGHT, IMG_WIDTH, 3)),
        tf.keras.layers.Conv2D(16, 3, padding='same', activation='relu'),
        tf.keras.layers.MaxPooling2D(),
        tf.keras.layers.Conv2D(32, 3, padding='same', activation='relu'),
        tf.keras.layers.MaxPooling2D(),
        tf.keras.layers.Conv2D(64, 3, padding='same', activation='relu'),
        tf.keras.layers.MaxPooling2D(),
        tf.keras.layers.Flatten(),
        tf.keras.layers.Dense(num_units, activation='relu'),
        tf.keras.layers.Dense(NUM_CLASSES, activation='softmax')
    ])

    model.compile(optimizer=tf.keras.optimizers.SGD(learning_rate=learning_rate,
momentum=momentum),
        loss=tf.keras.losses.SparseCategoricalCrossentropy(),
        metrics=['accuracy'])

    return model

def main():
    args = get_args()
    train_dataset, validation_dataset = create_datasets(DATA_DIR, BATCH_SIZE)
    model = create_model(args.num_units, args.learning_rate, args.momentum)
    history = model.fit(train_dataset, validation_data=validation_dataset, epochs=EPOCHS)

    # DEFINE METRIC
    hp_metric = history.history['val_accuracy'][-1]

    hpt = hypertune.HyperTune()
    hpt.report_hyperparameter_tuning_metric(
        hyperparameter_metric_tag='accuracy',
        metric_value=hp_metric,
        global_step=EPOCHS)

```



```
if __name__ == "__main__":  
    main()
```

建構容器前，請先深入瞭解程式碼。使用超參數調整服務時，有幾項專屬元件。

1. 指令碼會匯入 `hypertune` 程式庫。
2. 函式 `get_args()` 會為您要調整的每個超參數定義指令列引數。在本範例中，要調整的超參數是學習率、最佳化工具中的動量值，以及模型最後一個隱藏層中的單元數量，但您也可以嘗試其他超參數。然後，系統會使用這些引數中傳遞的值，在程式碼中設定對應的超參數。
3. 在 `main()` 函式結尾，系統會使用 `hypertune` 程式庫定義要最佳化的指標。在 TensorFlow 中，`keras model.fit` 方法會傳回 `History` 物件。`History.history` 屬性會記錄連續訓練週期的訓練損失值和指標值。如果將驗證資料傳遞至 `model.fit`，`History.history` 屬性也會包含驗證損失和指標值。舉例來說，如果您使用驗證資料訓練模型三個訓練週期，並提供 `accuracy` 做為指標，則 `History.history` 屬性會類似於下列字典。

```
{  
  "accuracy": [  
    0.7795261740684509,  
    0.9471358060836792,  
    0.9870933294296265  
  ],  
  "loss": [  
    0.6340447664260864,  
    0.16712145507335663,  
    0.04546636343002319  
  ],  
  "val_accuracy": [  
    0.3795261740684509,  
    0.4471358060836792,  
    0.4870933294296265  
  ],  
  "val_loss": [  
    2.044623374938965,  
    4.100203514099121,  
    3.0728273391723633  
  ]  
}
```

如要讓超參數調整服務找出可將模型驗證準確率提升至最高的超參數值，請將指標定義為 `val_accuracy` 清單的最後一個項目 (或 `NUM_EPOCHS - 1`)。然後將這項指標傳遞至 `HyperTune` 的執行個體。您可以為 `hyperparameter_metric_tag` 選擇任何字串，但稍後啟動超參數調整工作時，需要再次使用該字串。

步驟 2：建立 Dockerfile

如要將程式碼容器化，您需要建立 `Dockerfile`。`Dockerfile` 包含執行映像檔所需的所有指令。這會安裝所有必要的程式庫，並設定訓練程式碼的進入點。

在終端機中，於 `flowers-hptune` 目錄的根目錄中建立空白的 `Dockerfile`：

```
touch Dockerfile
```

flowers-hptune/ 目錄現在應該包含下列項目：

```
+ Dockerfile
+ trainer/
  + task.py
```

開啟 Dockerfile，然後將下列內容複製到其中。您會發現這與第一個實驗室中使用的 Dockerfile 幾乎相同，只是現在我們安裝了 cloudml-hypertune 程式庫。

```
FROM gcr.io/deeplearning-platform-release/tf2-gpu.2-8

WORKDIR /

# Installs hypertune library
RUN pip install cloudml-hypertune

# Copies the trainer code to the docker image.
COPY trainer /trainer

# Sets up the entry point to invoke the trainer.
ENTRYPOINT ["python", "-m", "trainer.task"]
```

步驟 3：建構容器

在終端機中執行下列指令，為專案定義環境變數，並將 `your-cloud-project` 替換為專案 ID：

您可以在終端機中執行 `gcloud config list --format 'value(core.project)'` 來取得專案 ID

```
PROJECT_ID='your-cloud-project'
```

在 Artifact Registry 中定義存放區。我們會使用在第一個實驗室中建立的存放區。

```
REPO_NAME='flower-app'
```

在 Google Artifact Registry 中，使用容器映像檔的 URI 定義變數：

```
IMAGE_URI=us-central1-docker.pkg.dev/$PROJECT_ID/$REPO_NAME/flower_image_hptune:latest
```

設定 Docker

```
gcloud auth configure-docker \
  us-central1-docker.pkg.dev
```

接著，從 `flower-hptune` 目錄的根層級執行下列指令，建構容器：

```
docker build ./ -t $IMAGE_URI
```

最後，將其推送至 Artifact Registry：

```
docker push $IMAGE_URI
```

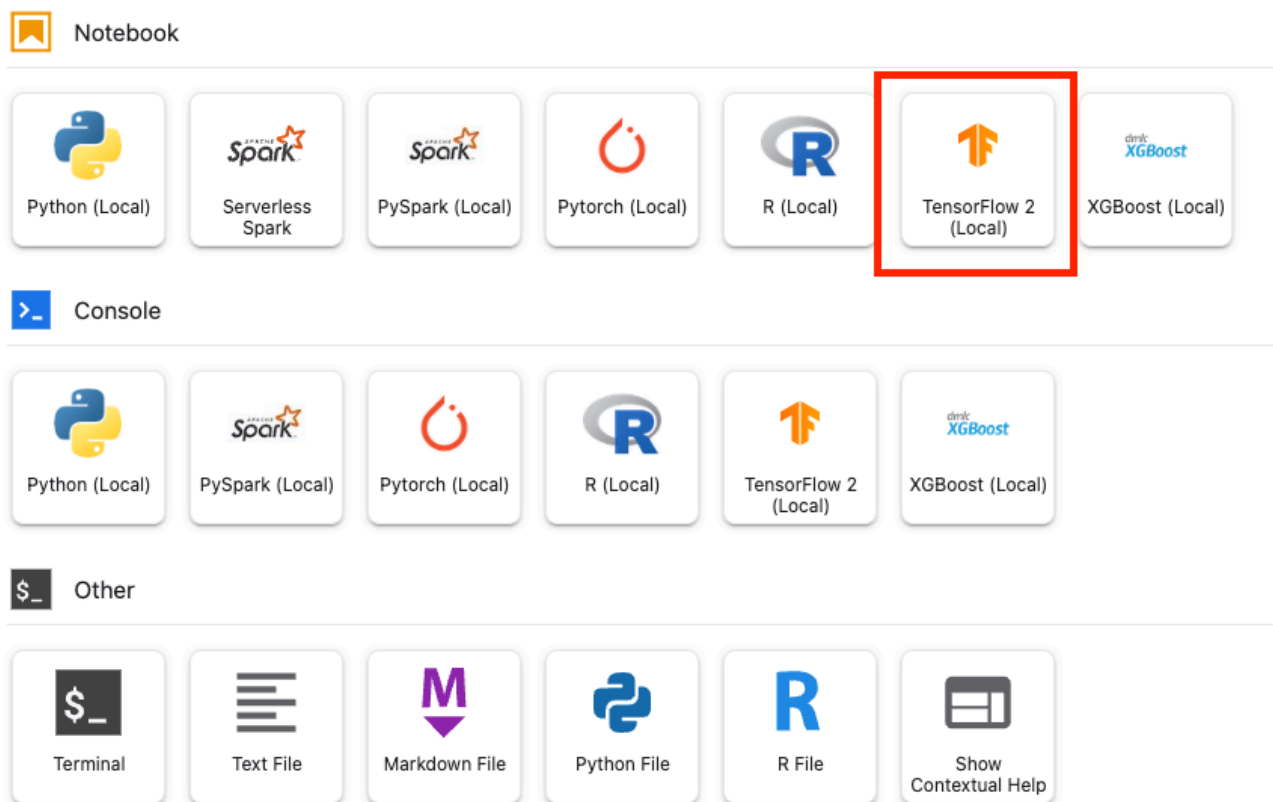
容器已推送至 Artifact Registry，現在可以啟動訓練工作。

步驟 5 · 約 30 分鐘

5. 使用 SDK 執行超參數調整工作

本節將說明如何使用 Vertex Python API 設定及提交超參數調整工作。

從 Launcher 建立 TensorFlow 2 筆記本。



匯入 Vertex AI SDK。

```
from google.cloud import aiplatform
from google.cloud.aiplatform import hyperparameter_tuning as hpt
```

如要啟動超參數調整工作，您必須先定義 `worker_pool_specs`，指定機型和 Docker 映像檔。以下規格定義了一部搭載兩個 NVIDIA Tesla V100 GPU 的機器。

您需要將 `image_uri` 中的 `{PROJECT_ID}` 換成您的專案。

```
# The spec of the worker pools including machine type and Docker image
# Be sure to replace PROJECT_ID in the `image_uri` with your project.

worker_pool_specs = [{
    "machine_spec": {
        "machine_type": "n1-standard-4",
        "accelerator_type": "NVIDIA_TESLA_V100",
        "accelerator_count": 1
    },
    "replica_count": 1,
    "container_spec": {
        "image_uri": "us-central1-docker.pkg.dev/{PROJECT_ID}/flower-
app/flower_image_hptune:latest"
    }
}]
```

接著定義 `parameter_spec`，這是指定要最佳化參數的字典。字典鍵是您為每個超參數指派的指令列引數字串，字典值則是參數規格。

您需要為每個超參數定義「類型」，以及微調服務會嘗試的值範圍。超參數可以是 `Double`、`Integer`、`Categorical` 或 `Discrete` 類型。如果選取「`Double`」或「`Integer`」類型，則必須提供最小值和最大值。如果選取「類別」或「離散」，則必須提供值。如果是「`Double`」和「`Integer`」類型，您也需要提供「`Scaling`」值。如要進一步瞭解如何選擇最佳比例，請觀看[這部影片](#)。

```
# Dictionary representing parameters to optimize.
# The dictionary key is the parameter_id, which is passed into your training
# job as a command line argument,
# And the dictionary value is the parameter specification of the metric.
parameter_spec = {
    "learning_rate": hpt.DoubleParameterSpec(min=0.001, max=1, scale="log"),
    "momentum": hpt.DoubleParameterSpec(min=0, max=1, scale="linear"),
    "num_units": hpt.DiscreteParameterSpec(values=[64, 128, 512], scale=None)
}
```

最後要定義的規格是 `metric_spec`，這是代表要最佳化指標的字典。字典鍵是您在訓練應用程式程式碼中設定的 `hyperparameter_metric_tag`，值則是最佳化目標。

```
# Dictionary representing metric to optimize.
# The dictionary key is the metric_id, which is reported by your training job,
# And the dictionary value is the optimization goal of the metric.
metric_spec={'accuracy': 'maximize'}
```

定義規格後，您將建立 `CustomJob`，這是用於在每個超參數調整試驗中執行工作的通用規格。

您需要將 `{YOUR_BUCKET}` 替換為先前建立的 bucket。

```
# Replace YOUR_BUCKET
my_custom_job = aiplatform.CustomJob(display_name='flowers-hptune-job',
                                     worker_pool_specs=worker_pool_specs,
                                     staging_bucket='gs://{YOUR_BUCKET}')
```

然後建立並執行 `HyperparameterTuningJob`。

```
hp_job = aiplatform.HyperparameterTuningJob(
    display_name='flowers-hptune-job',
    custom_job=my_custom_job,
    metric_spec=metric_spec,
    parameter_spec=parameter_spec,
    max_trial_count=15,
    parallel_trial_count=3)

hp_job.run()
```

請注意以下幾個引數：

- **max_trial_count**：您必須為服務執行的測試次數設定上限。一般來說，測試次數越多，結果越好，但會出現報酬遞減的情況，之後再增加測試次數，對您想盡力提升的指標幾乎沒有影響。建議您先進行少量試驗，瞭解所選超參數的影響程度，再擴大規模。
- **parallel_trial_count**：如果使用平行試驗，服務會佈建多個訓練處理叢集。增加平行試驗的數量可縮短超參數微調工作執行時間，但可能會降低整體工作成效。這是因為預設的微調策略會根據先前的試驗結果，在後續試驗中指派值。
- **search_algorithm**：您可以將搜尋演算法設為格線、隨機或預設（無）。預設選項會套用貝氏最佳化方法來搜尋可能的超參數值空間，是建議使用的演算法。如要進一步瞭解這項演算法，請參閱這篇文章。

您可以在控制台中查看工作進度。

Training + CREATE

TRAINING PIPELINES

CUSTOM JOBS

HYPERPARAMETER TUNING JOBS

Hyperparameter tuning searches for the best combination of hyperparameter values by optimising metric values across a series of trials. Hyperparameter tuning is only used by custom-trained models and not AutoML models. [Learn more](#)

Region

us-central1 (Iowa)

▼ ?

Filter

Enter a property name

Name	ID	Status	Job type	Model type
flowers-hptune-job	5418332003906879488	Running	Hyperparameter tuning job	—

完成後，您就能查看各項試驗的結果，以及成效最佳的值組合。

Hyperparameter tuning trials

Filter

Enter property name or value

☐	Trial ID	accuracy ↑	Training step	learning_rate	momentum	num_units	Log
☐	7	0.214	10	2.653115046907186e-2	1	64	View logs
☐	4	0.24	10	1.7177733883862592e-1	7.449606565135071e-1	64	View logs
☐	12	0.24	10	2.9691395166525733e-2	8.377780658030651e-1	64	View logs
☐	3	0.518	10	1.4950026383340348e-3	4.143626588263256e-1	128	View logs
☐	9	0.546	10	1.925167097370439e-2	4.219609427352633e-1	64	View logs
☐	2	0.598	10	7.180290204695781e-3	2.855836638936463e-1	64	View logs
☐	14	0.613	10	2.591778209055399e-2	6.597969577176989e-1	64	View logs
☐	10	0.621	10	5.377726911101266e-2	4.628837403188791e-12	512	View logs
☐	5	0.627	10	2.8303702557647553e-2	2.4000457359497013e-1	128	View logs
☐	1	0.631	10	3.162277660168379e-2	5e-1	128	View logs

Rows per page: 10 1 – 10 of 15 < >



您已瞭解如何使用 Vertex AI 執行下列作業：

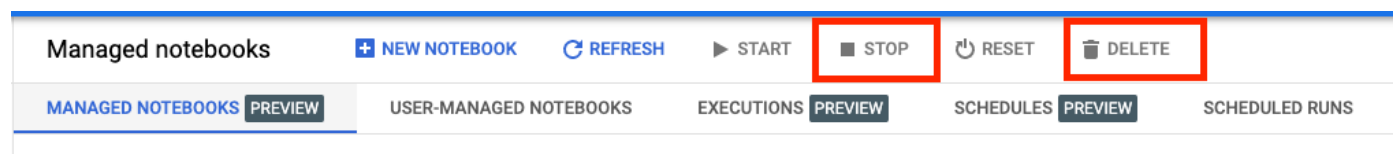
- 執行自動超參數調整工作

如要進一步瞭解 Vertex 的其他部分，請參閱[說明文件](#)。

步驟 6 · 約 1 分鐘

6. 清除所用資源

由於我們將筆記本設定為在閒置 60 分鐘後逾時，因此不必擔心執行個體會關閉。如要手動關閉執行個體，請按一下控制台 Vertex AI Workbench 專區的「停止」按鈕。如要徹底刪除筆記本，請按一下「刪除」按鈕。



如要刪除 Storage Bucket，請使用 Cloud 控制台中的導覽選單瀏覽至 Storage，選取 bucket，然後按一下「Delete」：

